

```
// File: server/index.js (// Lines: 62)
// =====
import dotenv from 'dotenv';
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
dotenv.config({ path: path.join(__dirname, '.env') });

import express from 'express';
import cors from 'cors';
import { connectDB } from './db/database.js';

import authRoutes from './routes/auth.js';
import productRoutes from './routes/products.js';
import pricingRoutes from './routes/pricing.js';
import couponRoutes from './routes/coupons.js';
import uploadRoutes from './routes/upload.js';
import orderRoutes from './routes/orders.js';
import reviewRoutes from './routes/reviews.js';
import dashboardRoutes from './routes/dashboard.js';
import addressRoutes from './routes/addresses.js';

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/pricing', pricingRoutes);
app.use('/api/coupons', couponRoutes);
app.use('/api/upload', uploadRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/reviews', reviewRoutes);
app.use('/api/dashboard', dashboardRoutes);
app.use('/api/addresses', addressRoutes);

app.get('/api/health', (req, res) => res.json({ status: 'ok' }));

const clientDistPath = path.join(__dirname, '..', 'client', 'dist');
app.use(express.static(clientDistPath));
app.get('*', (req, res) => {
  if (!req.path.startsWith('/api')) {
    res.sendFile(path.join(clientDistPath, 'index.html'));
  }
});

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal server error' });
});

async function start() {
  await connectDB();
  app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });
}

start();

// File: server/db/database.js (// Lines: 14)
// =====
import mongoose from 'mongoose';

let isConnected = false;

export async function connectDB() {
  if (isConnected) return;
  const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017/printshop';
  await mongoose.connect(uri);
  isConnected = true;
  console.log('MongoDB connected');
}

export default mongoose;

// File: server/db/seed.js (// Lines: 441)
// =====
import bcrypt from 'bcryptjs';
import mongoose from 'mongoose';
import User from '../models/User.js';
import Product from '../models/Product.js';
import PricingRule from '../models/PricingRule.js';
import FinishUpcharge from '../models/FinishUpcharge.js';
import Coupon from '../models/Coupon.js';
import Address from '../models/Address.js';
import Order from '../models/Order.js';
import OrderItem from '../models/OrderItem.js';
import OrderStatusHistory from '../models/OrderStatusHistory.js';
import Review from '../models/Review.js';
import Counter from '../models/Counter.js';

async function clearCollections() {

```

```

const collections = [
  Review, OrderStatusHistory, OrderItem, Order,
  FinishUpcharge, PricingRule, Coupon, Address, Product, User, Counter
];
for (const Model of collections) {
  await Model.deleteMany({});
}
}

async function seed() {
  const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017/printshop';
  await mongoose.connect(uri);
  console.log('Connected to MongoDB');

  await clearCollections();
  console.log('Seeding database...');

  const hashedPassword = bcrypt.hashSync('admin123', 10);
  const hashedCust = bcrypt.hashSync('cust123', 10);

  const admin = await User.create({ name: 'Admin User', email: 'admin@printshop.com', password: hashedPassword, phone: '9876543210', role: 'admin' });
  const custIds = [];
  for (let i = 1; i <= 5; i++) {
    const user = await User.create({ name: `Customer ${i}`, email: `customer${i}@printshop.com`, password: hashedCust, phone: `98765432${10 + i}`, role: 'customer' });
    custIds.push(user._id);
  }

  const poster1 = await Product.create({
    name: 'Premium Matte Poster', category: 'posters',
    description: 'High-quality matte finish posters printed on premium paper. Perfect for wall art, photography prints, and promotional displays. The matte finish reduces glare and gives a professional, elegant look.',
    base_price: 25, turnaround_time: '3-5 business days',
    available_sizes: ['A4', 'A3', 'A2', 'A1'],
    available_finishes: ['Matte', 'Glossy', 'Satin'],
    status: 'active', order_count: 85, avg_rating: 4.3, total_reviews: 4
  });

  const poster2 = await Product.create({
    name: 'Glossy Photo Poster', category: 'posters',
    description: 'Vibrant glossy photo posters with brilliant colour reproduction. Ideal for portraits, event posters, and marketing materials where colours need to pop. UV resistant coating protects against fading.',
    base_price: 28, turnaround_time: '3-5 business days',
    available_sizes: ['A4', 'A3', 'A2', 'A1'],
    available_finishes: ['Matte', 'Glossy', 'Satin'],
    status: 'active', order_count: 62, avg_rating: 4.5, total_reviews: 3
  });

  const poster3 = await Product.create({
    name: 'Satin Finish Art Print', category: 'posters',
    description: 'Custom shaped die-cut stickers made from high-quality vinyl. Available in various finishes including holographic and transparent. Perfect for branding, packaging, and personal projects. Waterproof option available.',
    base_price: 30, turnaround_time: '3-5 business days',
    available_sizes: ['A4', 'A3', 'A2', 'A1'],
    available_finishes: ['Matte', 'Glossy', 'Satin'],
    status: 'active', order_count: 48, avg_rating: 4.7, total_reviews: 3
  });

  const productPosters = [poster1._id, poster2._id, poster3._id];

  const sticker1 = await Product.create({
    name: 'Custom Die-Cut Stickers', category: 'stickers',
    description: 'Custom shaped die-cut stickers made from high-quality vinyl. Available in various finishes including holographic and transparent. Perfect for branding, packaging, and personal projects. Waterproof option available.',
    base_price: 12, turnaround_time: '5-7 business days',
    available_sizes: ['5x5cm', '7x7cm', '10x10cm'],
    available_finishes: ['Matte Laminate', 'Gloss Laminate', 'Transparent', 'Holographic'],
    status: 'active', order_count: 120, avg_rating: 4.6, total_reviews: 5
  });

  const sticker2 = await Product.create({
    name: 'Glossy Square Stickers', category: 'stickers',
    description: 'Bold glossy square stickers with vibrant print quality. Ideal for product labels, branding stickers, and promotional giveaways. Durable vinyl with strong adhesive backing.',
    base_price: 8, turnaround_time: '5-7 business days',
    available_sizes: ['5x5cm', '7x7cm', '10x10cm'],
    available_finishes: ['Matte Laminate', 'Gloss Laminate', 'Transparent', 'Holographic'],
    status: 'active', order_count: 95, avg_rating: 4.4, total_reviews: 3
  });

  const sticker3 = await Product.create({
    name: 'Holographic Circle Stickers', category: 'stickers',
    description: 'Eye-catching holographic circle stickers that shine and shimmer in light. Great for special editions, event stickers, and premium packaging. Each sticker has a unique rainbow effect.',
    base_price: 10, turnaround_time: '5-7 business days',
    available_sizes: ['5x5cm', '7x7cm', '10x10cm'],
    available_finishes: ['Matte Laminate', 'Gloss Laminate', 'Transparent', 'Holographic'],
    status: 'active', order_count: 73, avg_rating: 4.8, total_reviews: 4
  });

  const productStickers = [sticker1._id, sticker2._id, sticker3._id];

  const vc1 = await Product.create({
    name: 'Premium Business Cards', category: 'visiting-cards',
    description: 'Professional premium business cards on heavy 350gsm stock. Choose from multiple finishes including Soft Touch and Spot UV. Stand out with our premium quality card stock and precise cutting.',
    base_price: 3, turnaround_time: '4-6 business days',
    available_sizes: ['Standard (90x54mm)', 'Square (55x55mm)', 'Mini (85x28mm)'],

```

```

    available_finishes: ['Matte', 'Glossy', 'Soft Touch', 'Spot UV', 'Foil'],
    status: 'active', order_count: 156, avg_rating: 4.5, total_reviews: 5
  });

const vc2 = await Product.create({
  name: 'Luxury Spot UV Cards', category: 'visiting-cards',
  description: 'Make an impression with Spot UV coated business cards. The glossy UV coating highlights selected areas creating a striking contrast against the matte background. 400gsm premium stock.',
  base_price: 4, turnaround_time: '4-6 business days',
  available_sizes: ['Standard (90x54mm)', 'Square (55x55mm)', 'Mini (85x28mm)'],
  available_finishes: ['Matte', 'Glossy', 'Soft Touch', 'Spot UV', 'Foil'],
  status: 'active', order_count: 89, avg_rating: 4.6, total_reviews: 3
});

const vc3 = await Product.create({
  name: 'Foil Print Visiting Cards', category: 'visiting-cards',
  description: 'Luxurious foil printed business cards available in Gold or Silver. The metallic foil creates a premium raised effect that catches light beautifully. Choose from gold or silver foil on premium card stock.',
  base_price: 5, turnaround_time: '5-7 business days',
  available_sizes: ['Standard (90x54mm)', 'Square (55x55mm)', 'Mini (85x28mm)'],
  available_finishes: ['Matte', 'Glossy', 'Soft Touch', 'Spot UV', 'Foil'],
  status: 'active', order_count: 67, avg_rating: 4.9, total_reviews: 4
});

const productVC = [vc1._id, vc2._id, vc3._id];
const allProducts = [...productPosters, ...productStickers, ...productVC];

const pricingRulesData = [];
for (const pid of allProducts) {
  pricingRulesData.push(
    { product: pid, min_qty: 1, max_qty: 4, discount_percent: 0 },
    { product: pid, min_qty: 5, max_qty: 9, discount_percent: 10 },
    { product: pid, min_qty: 10, max_qty: 24, discount_percent: 18 },
    { product: pid, min_qty: 25, max_qty: 99999, discount_percent: 25 }
  );
}
await PricingRule.insertMany(pricingRulesData);

const upchargeData = [];
for (const pid of productPosters) {
  upchargeData.push(
    { product: pid, finish_name: 'Matte', upcharge_amount: 0 },
    { product: pid, finish_name: 'Glossy', upcharge_amount: 0 },
    { product: pid, finish_name: 'Satin', upcharge_amount: 5 }
  );
}
for (const pid of productStickers) {
  upchargeData.push(
    { product: pid, finish_name: 'Matte Laminate', upcharge_amount: 0 },
    { product: pid, finish_name: 'Gloss Laminate', upcharge_amount: 0 },
    { product: pid, finish_name: 'Transparent', upcharge_amount: 5 },
    { product: pid, finish_name: 'Holographic', upcharge_amount: 8 }
  );
}
for (const pid of productVC) {
  upchargeData.push(
    { product: pid, finish_name: 'Matte', upcharge_amount: 0 },
    { product: pid, finish_name: 'Glossy', upcharge_amount: 0 },
    { product: pid, finish_name: 'Soft Touch', upcharge_amount: 500 },
    { product: pid, finish_name: 'Spot UV', upcharge_amount: 300 },
    { product: pid, finish_name: 'Foil', upcharge_amount: 800 }
  );
}
await FinishUpcharge.insertMany(upchargeData);

await Coupon.insertMany([
  { code: 'WELCOME10', discount_type: 'percentage', discount_value: 10, min_order_amount: 500, max_uses: 100, usage_count: 15, expiry_date: new Date('2026-12-31'), is_active: true },
  { code: 'BULK20', discount_type: 'percentage', discount_value: 20, min_order_amount: 2000, max_uses: 50, usage_count: 8, expiry_date: new Date('2026-12-31'), is_active: true },
  { code: 'FLAT100', discount_type: 'flat', discount_value: 100, min_order_amount: 800, max_uses: 30, usage_count: 12, expiry_date: new Date('2026-12-31'), is_active: true },
]);

const cities = [
  { city: 'Mumbai', state: 'Maharashtra', pincode: '400001' },
  { city: 'Delhi', state: 'Delhi', pincode: '110001' },
  { city: 'Bangalore', state: 'Karnataka', pincode: '560001' },
  { city: 'Pune', state: 'Maharashtra', pincode: '411001' },
  { city: 'Chennai', state: 'Tamil Nadu', pincode: '600001' },
];
const streets = ['123 MG Road', '456 Park Street', '789 Brigade Road', '321 FC Road', '654 Anna Salai'];

const addresses = [];
for (let i = 0; i < custIds.length; i++) {
  addresses.push(await Address.create({
    user: custIds[i], label: 'Home', line1: streets[i], line2: 'Near Main Market',
    city: cities[i].city, state: cities[i].state, pincode: cities[i].pincode, is_default: true
  }));
}

const statuses = ['pending', 'confirmed', 'printing', 'shipped', 'delivered', 'cancelled'];
const paymentMethods = ['UPI', 'Net Banking', 'Credit Card', 'Debit Card', 'COD'];

const configurations = {
  posters: {
    base_config: { size: 'A4', finish: 'Matte', weight: '170gsm' },
    configs: [
      { size: 'A3', finish: 'Glossy', weight: '250gsm' },

```

```

    { size: 'A4', finish: 'Satin', weight: '170gsm' },
    { size: 'A2', finish: 'Matte', weight: '130gsm' },
    { size: 'A1', finish: 'Glossy', weight: '250gsm' },
    { size: 'A4', finish: 'Matte', weight: '170gsm' },
    { size: 'A3', finish: 'Satin', weight: '170gsm' },
  ],
},
stickers: {
  base_config: { type: 'Die-Cut', size: '7x7cm', finish: 'Matte Laminate', waterproof: false },
  configs: [
    { type: 'Square', size: '5x5cm', finish: 'Gloss Laminate', waterproof: true },
    { type: 'Circle', size: '10x10cm', finish: 'Holographic', waterproof: false },
    { type: 'Die-Cut', size: '7x7cm', finish: 'Transparent', waterproof: true },
    { type: 'Square', size: '10x10cm', finish: 'Gloss Laminate', waterproof: false },
    { type: 'Circle', size: '5x5cm', finish: 'Matte Laminate', waterproof: true },
  ]
},
'visiting-cards': {
  base_config: { size: 'Standard (90x54mm)', finish: 'Matte', thickness: '350gsm', corners: 'Rounded', sides: 'Double' },
  configs: [
    { size: 'Standard (90x54mm)', finish: 'Soft Touch', thickness: '400gsm', corners: 'Square', sides: 'Single' },
    { size: 'Square (55x55mm)', finish: 'Spot UV', thickness: '350gsm', corners: 'Rounded', sides: 'Double' },
    { size: 'Mini (85x28mm)', finish: 'Foil', thickness: '300gsm', corners: 'Square', sides: 'Single' },
    { size: 'Standard (90x54mm)', finish: 'Glossy', thickness: '350gsm', corners: 'Rounded', sides: 'Double' },
  ]
}
};

const catProducts = {
  posters: productPosters,
  stickers: productStickers,
  'visiting-cards': productVC
};

const productNames = {
  [poster1._id.toString()]: 'Premium Matte Poster',
  [poster2._id.toString()]: 'Glossy Photo Poster',
  [poster3._id.toString()]: 'Satin Finish Art Print',
  [sticker1._id.toString()]: 'Custom Die-Cut Stickers',
  [sticker2._id.toString()]: 'Glossy Square Stickers',
  [sticker3._id.toString()]: 'Holographic Circle Stickers',
  [vc1._id.toString()]: 'Premium Business Cards',
  [vc2._id.toString()]: 'Luxury Spot UV Cards',
  [vc3._id.toString()]: 'Foil Print Visiting Cards'
};

const pidToDoc = {};
for (const p of [poster1, poster2, poster3, sticker1, sticker2, sticker3, vc1, vc2, vc3]) {
  pidToDoc[p._id.toString()] = p;
}

const orderCounts = {};
for (const pid of allProducts) orderCounts[pid.toString()] = 0;

for (let i = 0; i < 25; i++) {
  const custIdx = i % custIds.length;
  const userId = custIds[custIdx];
  const createdAt = new Date(Date.now() - (25 - i) * 86400000 * (Math.random() * 0.5 + 0.5));
  const dateStr = createdAt.toISOString().slice(0, 10).replace(/-/g, '');
  const seq = String(i + 1).padStart(3, '0');
  const orderNumber = `ORD-${dateStr}-${seq}`;

  const cats = ['posters', 'stickers', 'visiting-cards'];
  const cat = cats[i % 3];
  const catProdArr = catProducts[cat];
  const catConfig = configurations[cat];
  const items = [];
  let subtotal = 0;
  const numItems = (i % 3) + 1;

  const allOrderItems = [];

  for (let j = 0; j < numItems; j++) {
    const pid = catProdArr[(i + j) % catProdArr.length];
    const qty = [5, 10, 25, 50, 100][i % 5];
    const configIdx = j % catConfig.configs.length;
    const cfg = { ...catConfig.configs[configIdx], quantity: qty };

    const product = pidToDoc[pid.toString()];
    const basePrice = product.base_price;

    const rules = await PricingRule.find({ product: pid, min_qty: { $lte: qty }, max_qty: { $gte: qty } });
    let discountPercent = rules.length > 0 ? rules[0].discount_percent : 0;

    const upchargeRows = await FinishUpcharge.find({ product: pid, finish_name: cfg.finish });
    const upcharge = upchargeRows.length > 0 ? upchargeRows[0].upcharge_amount : 0;

    let effectiveUnitPrice = basePrice + upcharge;
    if (cat === 'stickers' && cfg.waterproof) {
      effectiveUnitPrice = effectiveUnitPrice * 1.1;
    }
    if (cat === 'visiting-cards') {
      if (cfg.sides === 'Double') effectiveUnitPrice += 200 / qty;
      if (cfg.corners === 'Rounded') effectiveUnitPrice += 150 / qty;
    }

    const discountedPrice = effectiveUnitPrice * (1 - discountPercent / 100);
    const lineTotal = Math.round(discountedPrice * qty * 100) / 100;
    subtotal += lineTotal;
  }
}

```

```

orderCounts[pid.toString()] = (orderCounts[pid.toString()] || 0) + qty;

allOrderItems.push({
  pid,
  name: productNames[pid.toString()],
  category: cat,
  configuration: cfg,
  design_file_path: '',
  design_notes: '',
  no_design_flag: i % 3 === 0,
  quantity: qty,
  unit_price: Math.round(discountedPrice * 100) / 100,
  discount_applied: Math.round((effectiveUnitPrice - discountedPrice) * 100) / 100,
  line_total: lineTotal
});
}

let couponCode = '';
let couponDiscount = 0;
let gstAmount = Math.round(subtotal * 0.18 * 100) / 100;
let grandTotal = subtotal + gstAmount;

if (i % 5 === 0 && subtotal >= 500) {
  couponCode = 'WELCOME10';
  couponDiscount = Math.round(subtotal * 0.1 * 100) / 100;
  const afterCoupon = subtotal - couponDiscount;
  gstAmount = Math.round(afterCoupon * 0.18 * 100) / 100;
  grandTotal = afterCoupon + gstAmount;
}

const status = statuses[i % statuses.length];
const paymentMethod = paymentMethods[i % paymentMethods.length];
const customerName = `Customer ${custIdx + 1}`;
const customerEmail = `customer${custIdx + 1}@printshop.com`;
const customerPhone = `98765432${10 + custIdx}`;
const deliveryAddress = {
  line1: streets[custIdx],
  line2: 'Near Main Market',
  city: cities[custIdx].city,
  state: cities[custIdx].state,
  pincode: cities[custIdx].pincode
};

const order = await Order.create({
  order_number: orderNumber,
  user: userId,
  customer_name: customerName,
  customer_email: customerEmail,
  customer_phone: customerPhone,
  delivery_address: deliveryAddress,
  order_notes: i % 4 === 0 ? 'Please handle with care. Rush order.' : '',
  subtotal,
  coupon_code: couponCode,
  coupon_discount: couponDiscount,
  gst_amount: gstAmount,
  grand_total: grandTotal,
  payment_method: paymentMethod,
  payment_status: 'completed',
  status,
  internal_notes: '',
  created_at: createdAt,
  updated_at: createdAt,
});

const orderItemDocs = [];
for (const item of allOrderItems) {
  const oi = await OrderItem.create({
    order: order._id,
    product: item.pid,
    product_name: item.name,
    category: item.category,
    configuration: item.configuration,
    design_file_path: item.design_file_path,
    design_notes: item.design_notes,
    no_design_flag: item.no_design_flag,
    quantity: item.quantity,
    unit_price: item.unit_price,
    discount_applied: item.discount_applied,
    line_total: item.line_total,
  });
  orderItemDocs.push(oi);
}

const statusFlow = ['pending', 'confirmed', 'printing', 'shipped', 'delivered'];
const statusIdx = statusFlow.indexOf(status);
if (statusIdx >= 0) {
  await OrderStatusHistory.create({ order: order._id, status: 'pending', changed_by: 'system', notes: 'Order placed', created_at: create
dAt });
  for (let s = 1; s <= statusIdx; s++) {
    const stDate = new Date(createdAt.getTime() + s * 86400000);
    await OrderStatusHistory.create({ order: order._id, status: statusFlow[s], changed_by: 'system', notes: `Status updated to ${statusF
low[s]}`, created_at: stDate });
  }
} else if (status === 'cancelled') {
  await OrderStatusHistory.create({ order: order._id, status: 'pending', changed_by: 'system', notes: 'Order placed', created_at: create
dAt });
  await OrderStatusHistory.create({ order: order._id, status: 'cancelled', changed_by: 'customer', notes: 'Cancelled by customer', creat
ed_at: new Date(createdAt.getTime() + 86400000) });
}

```

```

    }
  }

  for (const [pidStr, count] of Object.entries(orderCounts)) {
    await Product.findByIdAndUpdate(pidStr, { order_count: count });
  }

  const deliveredOrders = await Order.find({ status: 'delivered' }).select('_id');
  const deliveredItems = await OrderItem.find({ order: { $in: deliveredOrders.map(o => o._id) } }).populate('order');
  const deliveredItemIds = deliveredItems.filter(di => di.order?.status === 'delivered');

  const reviewComments = [
    5: ['Absolutely stunning print quality! Highly recommended.', 'Perfect finish and fast delivery. Will order again.', 'Exceeded my expectations. The colours are vibrant!'],
    4: ['Great quality for the price. Very satisfied.', 'Good print quality. Delivery was on time.', 'Nice product. Would recommend to others.'],
    3: ['Decent quality but could be better.', 'Okay product for the price point.'],
  ];

  for (let i = 0; i < Math.min(12, deliveredItemIds.length); i++) {
    const item = deliveredItemIds[i];
    const rating = [5, 5, 4, 5, 4, 3, 5, 4, 5, 4, 5, 5][i];
    const comments = reviewComments[rating] || ['Great product!'];
    const comment = comments[i % comments.length];
    const userId = custIds[i % custIds.length];
    await Review.create({ product: item.product, user: userId, order_item: item._id, rating, comment });

    const stats = await Review.aggregate([
      { $match: { product: item.product } },
      { $group: { _id: null, avg: { $avg: '$rating' }, count: { $sum: 1 } } }
    ]);
    if (stats.length > 0) {
      await Product.findByIdAndUpdate(item.product, {
        avg_rating: Math.round(stats[0].avg * 10) / 10,
        total_reviews: stats[0].count,
      });
    }
  }

  console.log('Database seeded successfully!');
  console.log('Admin: admin@printshop.com / admin123');
  console.log('Customer: customer1@printshop.com / cust123');
  process.exit(0);
}

```

```

seed().catch(err => {
  console.error('Seed failed:', err);
  process.exit(1);
});

```

// File: server/middleware/auth.js (// Lines: 24)

```

// =====
import jwt from 'jsonwebtoken';

export function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }
  try {
    const token = header.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

export function requireAdmin(req, res, next) {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required' });
  }
  next();
}

```

// File: server/middleware/upload.js (// Lines: 31)

```

// =====
import multer from 'multer';
import path from 'path';
import fs from 'fs';

const uploadDir = process.env.UPLOAD_DIR || 'uploads';
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const ts = Date.now();
    const ext = path.extname(file.originalname);
    const safeName = file.originalname.replace(ext, '').replace(/^[a-zA-Z0-9_-]/g, '_');
    cb(null, `${ts}-${safeName}${ext}`);
  }
});

```

```

const fileFilter = (req, file, cb) => {
  const allowed = ['.pdf', '.png', '.jpg', '.jpeg', '.ai', '.eps', '.svg', '.psd', '.tiff'];
  const ext = path.extname(file.originalname).toLowerCase();
  if (allowed.includes(ext)) {
    cb(null, true);
  } else {
    cb(new Error('Only PDF, PNG, JPG, AI, EPS, SVG, PSD, TIFF files are allowed'), false);
  }
};

export const upload = multer({ storage, fileFilter, limits: { fileSize: 20 * 1024 * 1024 } });

// File: server/models/Address.js (// Lines: 15)
// =====
import mongoose from 'mongoose';

const addressSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  label: { type: String, default: 'Home' },
  line1: { type: String, required: true },
  line2: { type: String, default: '' },
  city: { type: String, required: true },
  state: { type: String, required: true },
  pincode: { type: String, required: true },
  is_default: { type: Boolean, default: false },
}, { timestamps: { createdAt: 'created_at', updatedAt: false } });

export default mongoose.model('Address', addressSchema);

// File: server/models/Counter.js (// Lines: 20)
// =====
import mongoose from 'mongoose';

const counterSchema = new mongoose.Schema({
  _id: { type: String, required: true },
  seq: { type: Number, default: 0 },
});

const Counter = mongoose.model('Counter', counterSchema);

export async function getNextSequence(name) {
  const counter = await Counter.findByIdAndUpdate(
    name,
    { $inc: { seq: 1 } },
    { upsert: true, returnDocument: 'after' }
  );
  return counter.seq;
}

export default Counter;

// File: server/models/Coupon.js (// Lines: 15)
// =====
import mongoose from 'mongoose';

const couponSchema = new mongoose.Schema({
  code: { type: String, required: true, unique: true, uppercase: true },
  discount_type: { type: String, required: true, enum: ['percentage', 'flat'] },
  discount_value: { type: Number, required: true },
  min_order_amount: { type: Number, default: 0 },
  max_uses: { type: Number, default: 0 },
  usage_count: { type: Number, default: 0 },
  expiry_date: { type: Date, default: null },
  is_active: { type: Boolean, default: true },
}, { timestamps: { createdAt: 'created_at', updatedAt: false } });

export default mongoose.model('Coupon', couponSchema);

// File: server/models/FinishUpcharge.js (// Lines: 10)
// =====
import mongoose from 'mongoose';

const finishUpchargeSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  finish_name: { type: String, required: true },
  upcharge_amount: { type: Number, default: 0 },
});

export default mongoose.model('FinishUpcharge', finishUpchargeSchema);

// File: server/models/Order.js (// Lines: 23)
// =====
import mongoose from 'mongoose';

const orderSchema = new mongoose.Schema({
  order_number: { type: String, required: true, unique: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
  customer_name: { type: String, required: true },
  customer_email: { type: String, required: true },
  customer_phone: { type: String, required: true },
  delivery_address: { type: mongoose.Schema.Types.Mixed, required: true },
  order_notes: { type: String, default: '' },
  subtotal: { type: Number, required: true },
  coupon_code: { type: String, default: '' },
  coupon_discount: { type: Number, default: 0 },

```

```

    gst_amount: { type: Number, required: true },
    grand_total: { type: Number, required: true },
    payment_method: { type: String, required: true },
    payment_status: { type: String, default: 'completed' },
    status: { type: String, enum: ['pending', 'confirmed', 'printing', 'shipped', 'delivered', 'cancelled'], default: 'pending' },
    internal_notes: { type: String, default: '' },
  }, { timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' } });

export default mongoose.model('Order', orderSchema);

// File: server/models/OrderItem.js (// Lines: 19)
// =====
import mongoose from 'mongoose';

const orderItemSchema = new mongoose.Schema({
  order: { type: mongoose.Schema.Types.ObjectId, ref: 'Order', required: true },
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', default: null },
  product_name: { type: String, required: true },
  category: { type: String, required: true },
  configuration: { type: mongoose.Schema.Types.Mixed, required: true },
  design_file_path: { type: String, default: '' },
  design_notes: { type: String, default: '' },
  no_design_flag: { type: Boolean, default: false },
  quantity: { type: Number, required: true },
  unit_price: { type: Number, required: true },
  discount_applied: { type: Number, default: 0 },
  line_total: { type: Number, required: true },
});

export default mongoose.model('OrderItem', orderItemSchema);

// File: server/models/OrderStatusHistory.js (// Lines: 11)
// =====
import mongoose from 'mongoose';

const orderStatusHistorySchema = new mongoose.Schema({
  order: { type: mongoose.Schema.Types.ObjectId, ref: 'Order', required: true },
  status: { type: String, required: true },
  changed_by: { type: String, default: 'system' },
  notes: { type: String, default: '' },
}, { timestamps: { createdAt: 'created_at', updatedAt: false } });

export default mongoose.model('OrderStatusHistory', orderStatusHistorySchema);

// File: server/models/PricingRule.js (// Lines: 11)
// =====
import mongoose from 'mongoose';

const pricingRuleSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  min_qty: { type: Number, required: true },
  max_qty: { type: Number, required: true },
  discount_percent: { type: Number, required: true },
});

export default mongoose.model('PricingRule', pricingRuleSchema);

// File: server/models/Product.js (// Lines: 18)
// =====
import mongoose from 'mongoose';

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  category: { type: String, required: true, enum: ['posters', 'stickers', 'visiting-cards'] },
  description: { type: String, default: '' },
  base_price: { type: Number, required: true },
  turnaround_time: { type: String, default: '3-5 business days' },
  available_sizes: { type: [String], default: [] },
  available_finishes: { type: [String], default: [] },
  status: { type: String, enum: ['active', 'inactive'], default: 'active' },
  order_count: { type: Number, default: 0 },
  avg_rating: { type: Number, default: 0 },
  total_reviews: { type: Number, default: 0 },
}, { timestamps: { createdAt: 'created_at', updatedAt: false } });

export default mongoose.model('Product', productSchema);

// File: server/models/Review.js (// Lines: 12)
// =====
import mongoose from 'mongoose';

const reviewSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  order_item: { type: mongoose.Schema.Types.ObjectId, ref: 'OrderItem', default: null },
  rating: { type: Number, required: true, min: 1, max: 5 },
  comment: { type: String, default: '' },
}, { timestamps: { createdAt: 'created_at', updatedAt: false } });

export default mongoose.model('Review', reviewSchema);

// File: server/models/User.js (// Lines: 12)
// =====
import mongoose from 'mongoose';

```



```

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  phone: { type: String, default: '' },
  role: { type: String, enum: ['customer', 'admin'], default: 'customer' },
}, { timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' } });

export default mongoose.model('User', userSchema);

// File: server/controllers/addressController.js (// Lines: 59)
// =====
import Address from '../models/Address.js';

export async function listAddresses(req, res) {
  try {
    const addresses = await Address.find({ user: req.user.id }).sort({ is_default: -1, created_at: -1 });
    res.json(addresses);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch addresses' });
  }
}

export async function createAddress(req, res) {
  try {
    const { label, line1, line2, city, state, pincode, is_default } = req.body;
    if (is_default) {
      await Address.updateMany({ user: req.user.id }, { is_default: false });
    }
    const addr = await Address.create({
      user: req.user.id,
      label: label || 'Home',
      line1,
      line2: line2 || '',
      city,
      state,
      pincode,
      is_default: is_default || false,
    });
    res.status(201).json({ id: addr._id.toString(), message: 'Address created' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to create address' });
  }
}

export async function updateAddress(req, res) {
  try {
    const addr = await Address.findOne({ _id: req.params.id, user: req.user.id });
    if (!addr) return res.status(404).json({ error: 'Address not found' });
    const { label, line1, line2, city, state, pincode, is_default } = req.body;
    if (is_default) {
      await Address.updateMany({ user: req.user.id }, { is_default: false });
    }
    await Address.findByIdAndUpdate(req.params.id, { label, line1, line2: line2 || '', city, state, pincode, is_default: is_default || false });
    res.json({ message: 'Address updated' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update address' });
  }
}

export async function deleteAddress(req, res) {
  try {
    const addr = await Address.findOne({ _id: req.params.id, user: req.user.id });
    if (!addr) return res.status(404).json({ error: 'Address not found' });
    await Address.findByIdAndDelete(req.params.id);
    res.json({ message: 'Address deleted' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to delete address' });
  }
}

// File: server/controllers/authController.js (// Lines: 60)
// =====
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

export async function register(req, res) {
  try {
    const { name, email, password, phone } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ error: 'Name, email and password are required' });
    }
    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) {
      return res.status(400).json({ error: 'Email already registered' });
    }
    const hashedPassword = bcrypt.hashSync(password, 10);
    const user = await User.create({ name, email: email.toLowerCase(), password: hashedPassword, phone: phone || '', role: 'customer' });
    const token = jwt.sign({ id: user._id.toString(), email: user.email, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.status(201).json({ token, user: { id: user._id.toString(), name: user.name, email: user.email, phone: user.phone || '', role: user.role } });
  } catch (err) {
    if (err.code === 11000) {
      return res.status(400).json({ error: 'Email already registered' });
    }
  }
}

```

```

    res.status(500).json({ error: 'Registration failed' });
  }
}

export async function login(req, res) {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ error: 'Email and password are required' });
    }
    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }
    const valid = bcrypt.compareSync(password, user.password);
    if (!valid) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }
    const token = jwt.sign({ id: user._id.toString(), email: user.email, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.json({ token, user: { id: user._id.toString(), name: user.name, email: user.email, phone: user.phone, role: user.role } });
  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({ error: 'Login failed' });
  }
}

export async function getMe(req, res) {
  try {
    const user = await User.findById(req.user.id).select('id name email phone role created_at');
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.json(user.toObject());
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch user' });
  }
}

// File: server/controllers/couponController.js (// Lines: 97)
// =====
import Coupon from '../models/Coupon.js';

export async function validateCoupon(req, res) {
  try {
    const { code, subtotal } = req.body;
    if (!code) return res.status(400).json({ error: 'Coupon code is required' });
    const coupon = await Coupon.findOne({ code: code.toUpperCase() });
    if (!coupon) return res.status(404).json({ error: 'Invalid coupon code' });
    if (!coupon.is_active) return res.status(400).json({ error: 'Coupon is inactive' });
    if (coupon.expiry_date && new Date(coupon.expiry_date) < new Date()) {
      return res.status(400).json({ error: 'Coupon has expired' });
    }
    if (coupon.max_uses > 0 && coupon.usage_count >= coupon.max_uses) {
      return res.status(400).json({ error: 'Coupon usage limit reached' });
    }
    const orderTotal = subtotal || 0;
    if (orderTotal < coupon.min_order_amount) {
      return res.status(400).json({ error: `Minimum order amount of ${coupon.min_order_amount}` });
    }
    let discount = 0;
    if (coupon.discount_type === 'percentage') {
      discount = Math.round((orderTotal * coupon.discount_value / 100 * 100) / 100);
      if (discount > orderTotal) discount = orderTotal;
    } else {
      discount = Math.min(coupon.discount_value, orderTotal);
    }
    res.json({
      valid: true,
      code: coupon.code,
      discount_type: coupon.discount_type,
      discount_value: coupon.discount_value,
      discount,
      description: coupon.discount_type === 'percentage'
        ? `${coupon.discount_value}% off (max $${discount})`
        : `${coupon.discount_value} flat off`
    });
  } catch (err) {
    res.status(500).json({ error: 'Failed to validate coupon' });
  }
}

export async function listCoupons(req, res) {
  try {
    const coupons = await Coupon.find().sort({ created_at: -1 });
    res.json(coupons);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch coupons' });
  }
}

export async function createCoupon(req, res) {
  try {
    const { code, discount_type, discount_value, min_order_amount, max_uses, expiry_date, is_active } = req.body;
    await Coupon.create({
      code: code.toUpperCase(),
      discount_type,
      discount_value: Number(discount_value),
      min_order_amount: Number(min_order_amount || 0),
    });
  }
}

```

```

        max_uses: Number(max_uses || 0),
        expiry_date: expiry_date || null,
        is_active: is_active !== undefined ? is_active : true,
    });
    res.status(201).json({ message: 'Coupon created' });
} catch (err) {
    if (err.code === 11000) {
        return res.status(400).json({ error: 'Coupon code already exists' });
    }
    res.status(500).json({ error: 'Failed to create coupon' });
}
}

export async function updateCoupon(req, res) {
    try {
        const { discount_type, discount_value, min_order_amount, max_uses, expiry_date, is_active } = req.body;
        await Coupon.findByIdAndUpdate(req.params.id, {
            discount_type,
            discount_value: Number(discount_value),
            min_order_amount: Number(min_order_amount || 0),
            max_uses: Number(max_uses || 0),
            expiry_date: expiry_date || null,
            is_active: is_active !== undefined ? is_active : true,
        });
        res.json({ message: 'Coupon updated' });
    } catch (err) {
        res.status(500).json({ error: 'Failed to update coupon' });
    }
}

export async function deleteCoupon(req, res) {
    try {
        await Coupon.findByIdAndDelete(req.params.id);
        res.json({ message: 'Coupon deleted' });
    } catch (err) {
        res.status(500).json({ error: 'Failed to delete coupon' });
    }
}

```

// File: server/controllers/dashboardController.js (// Lines: 121)

// =====

```

import Order from '../models/Order.js';
import User from '../models/User.js';
import OrderItem from '../models/OrderItem.js';

export async function getAdminDashboard(req, res) {
    try {
        const now = new Date();
        const todayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
        const thirtyDaysAgo = new Date(now.getTime() - 30 * 24 * 60 * 60 * 1000);

        const [
            ordersToday,
            revenueToday,
            pendingOrders,
            totalCustomers,
            totalOrdersAgg,
            totalRevenueAgg,
            dailyRevenue,
            ordersByCategory,
            ordersByStatus,
            topProducts,
            recentOrders
        ] = await Promise.all([
            Order.countDocuments({ created_at: { $gte: todayStart } }),
            Order.aggregate([
                { $match: { created_at: { $gte: todayStart }, status: { $ne: 'cancelled' } } },
                { $group: { _id: null, total: { $sum: '$grand_total' } } }
            ]),
            Order.countDocuments({ status: 'pending' }),
            User.countDocuments({ role: 'customer' }),
            Order.countDocuments({}),
            Order.aggregate([
                { $match: { status: { $ne: 'cancelled' } } },
                { $group: { _id: null, total: { $sum: '$grand_total' } } }
            ]),
            Order.aggregate([
                { $match: { created_at: { $gte: thirtyDaysAgo }, status: { $ne: 'cancelled' } } },
                { $group: {
                    _id: { $dateToString: { format: '%Y-%m-%d', date: '$created_at' } },
                    revenue: { $sum: '$grand_total' },
                    orders_count: { $sum: 1 }
                } },
                { $sort: { _id: 1 } },
                { $project: { date: '$_id', revenue: 1, orders_count: 1, _id: 0 } }
            ]),
            OrderItem.aggregate([
                { $lookup: { from: 'orders', localField: 'order', foreignField: '_id', as: 'order' } },
                { $unwind: '$order' },
                { $match: { 'order.status': { $ne: 'cancelled' } } },
                { $group: { _id: '$category', count: { $sum: 1 }, total: { $sum: '$line_total' } } },
                { $sort: { count: -1 } },
                { $project: { category: '$_id', count: 1, total: 1, _id: 0 } }
            ]),
            Order.aggregate([
                { $group: { _id: '$status', count: { $sum: 1 } } },
                { $sort: { count: -1 } },
                { $project: { status: '$_id', count: 1, _id: 0 } }
            ]),

```

```

    }},
    OrderItem.aggregate([
      { $lookup: { from: 'products', localField: 'product', foreignField: '_id', as: 'product' } },
      { $unwind: '$product' },
      { $lookup: { from: 'orders', localField: 'order', foreignField: '_id', as: 'order' } },
      { $unwind: '$order' },
      { $match: { 'order.status': { $ne: 'cancelled' } } },
      { $group: {
        _id: '$product._id',
        name: { $first: '$product.name' },
        category: { $first: '$product.category' },
        total_qty: { $sum: '$quantity' },
        order_count: { $addToSet: '$order' }
      }},
      { $project: {
        id: '$_id',
        name: 1,
        category: 1,
        total_qty: 1,
        order_count: { $size: '$order_count' }
      }},
      { $sort: { total_qty: -1 } },
      { $limit: 5 }
    ]),
    Order.aggregate([
      { $lookup: { from: 'orderitems', localField: '_id', foreignField: 'order', as: 'items' } },
      { $sort: { created_at: -1 } },
      { $limit: 10 },
      { $project: {
        id: '$_id',
        order_number: 1,
        customer_name: 1,
        grand_total: 1,
        status: 1,
        payment_method: 1,
        created_at: 1,
        items_count: { $size: '$items' }
      }},
      { $limit: 10 }
    ]),
  });

res.json({
  summary: {
    orders_today: ordersToday,
    revenue_today: revenueToday[0]?.total || 0,
    pending_orders: pendingOrders,
    total_customers: totalCustomers,
    total_orders: totalOrdersAgg,
    total_revenue: totalRevenueAgg[0]?.total || 0
  },
  charts: {
    daily_revenue: dailyRevenue,
    orders_by_category: ordersByCategory,
    orders_by_status: ordersByStatus,
    top_products: topProducts
  },
  recent_orders: recentOrders
});
} catch (err) {
  console.error('Dashboard error:', err);
  res.status(500).json({ error: 'Failed to fetch dashboard data' });
}
}

// File: server/controllers/orderController.js (// Lines: 265)
// =====
import Order from '../models/Order.js';
import OrderItem from '../models/OrderItem.js';
import OrderStatusHistory from '../models/OrderStatusHistory.js';
import Product from '../models/Product.js';
import Coupon from '../models/Coupon.js';
import Counter from '../models/Counter.js';
import { jsPDF } from 'jspdf';

export async function createOrder(req, res) {
  try {
    const { items, customer_name, customer_email, customer_phone, delivery_address, order_notes, subtotal, coupon_code, coupon_discount, payment_method } = req.body;
    if (!items || !items.length) return res.status(400).json({ error: 'Cart is empty' });
    const gstAmount = Math.round((subtotal - (coupon_discount || 0)) * 0.18 * 100) / 100;
    const grandTotal = Math.round((subtotal - (coupon_discount || 0) + gstAmount) * 100) / 100;
    const today = new Date();
    const dateStr = today.toISOString().slice(0, 10).replace(/-/g, '');
    const counter = await Counter.findByIdAndUpdate(
      { _id: `order_${dateStr}` },
      { $inc: { seq: 1 } },
      { upsert: true, returnDocument: 'after' }
    );
    const seq = String(counter.seq).padStart(3, '0');
    const orderNumber = `ORD-${dateStr}-${seq}`;
    const addressStr = typeof delivery_address === 'string' ? delivery_address : JSON.stringify(delivery_address);

    const order = await Order.create({
      order_number: orderNumber,
      user: req.user?.id || null,
      customer_name,
      customer_email,
      customer_phone,

```

```

    delivery_address: addressStr,
    order_notes: order_notes || '',
    subtotal,
    coupon_code: coupon_code || '',
    coupon_discount: coupon_discount || 0,
    gst_amount: gstAmount,
    grand_total: grandTotal,
    payment_method,
    payment_status: 'completed',
    status: 'pending',
    internal_notes: '',
  });

const orderId = order._id;
const insertItemPromises = items.map(item =>
  OrderItem.create({
    order: orderId,
    product: item.product_id || null,
    product_name: item.product_name,
    category: item.category,
    configuration: item.configuration,
    design_file_path: item.design_file_path || '',
    design_notes: item.design_notes || '',
    no_design_flag: item.no_design_flag || false,
    quantity: item.quantity,
    unit_price: item.unit_price,
    discount_applied: item.discount_applied || 0,
    line_total: item.line_total,
  })
);
await Promise.all(insertItemPromises);

const productUpdatePromises = items.map(item =>
  Product.findByIdAndUpdate(item.product_id, { $inc: { order_count: item.quantity } })
);
await Promise.all(productUpdatePromises);

await OrderStatusHistory.create({
  order: orderId,
  status: 'pending',
  changed_by: 'customer',
  notes: 'Order placed',
});

if (coupon_code) {
  await Coupon.findOneAndUpdate({ code: coupon_code }, { $inc: { usage_count: 1 } });
}

res.status(201).json({
  order_id: orderId.toString(),
  order_number: orderNumber,
  grand_total: grandTotal,
  gst_amount: gstAmount,
  coupon_discount: coupon_discount || 0
});
} catch (err) {
  console.error('Order creation error:', err);
  res.status(500).json({ error: 'Failed to create order' });
}
}

export async function getMyOrders(req, res) {
  try {
    const orders = await Order.aggregate([
      { $match: { user: req.user.id } },
      { $lookup: { from: 'orderitems', localField: '_id', foreignField: 'order', as: 'items' } },
      { $addFields: { items_count: { $size: '$items' } } },
      { $project: { items: 0 } },
      { $sort: { created_at: -1 } }
    ]);
    res.json(orders);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch orders' });
  }
}

export async function getOrder(req, res) {
  try {
    const order = await Order.findById(req.params.id);
    if (!order) return res.status(404).json({ error: 'Order not found' });
    if (req.user.role !== 'admin' && order.user?.toString() !== req.user.id) {
      return res.status(403).json({ error: 'Access denied' });
    }
    const [items, statusHistory] = await Promise.all([
      OrderItem.find({ order: order._id }),
      OrderStatusHistory.find({ order: order._id }).sort({ created_at: 1 })
    ]);
    for (const item of items) {
      if (typeof item.configuration === 'string') {
        try { item.configuration = JSON.parse(item.configuration); } catch (e) { }
      }
    }
    let deliveryAddress = order.delivery_address;
    if (typeof deliveryAddress === 'string') {
      try { deliveryAddress = JSON.parse(deliveryAddress); } catch (e) { }
    }
    res.json({ ...order.toObject(), delivery_address: deliveryAddress, items, status_history: statusHistory });
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch order' });
  }
}

```

```

    }
  }

export async function listAllOrders(req, res) {
  try {
    const { status, search, dateFrom, dateTo } = req.query;
    const match = {};
    if (status) match.status = status;
    if (search) {
      match.$or = [
        { order_number: { $regex: search, $options: 'i' } },
        { customer_name: { $regex: search, $options: 'i' } }
      ];
    }
    if (dateFrom || dateTo) {
      match.created_at = {};
      if (dateFrom) match.created_at.$gte = new Date(dateFrom);
      if (dateTo) match.created_at.$lte = new Date(dateTo + 'T23:59:59.999Z');
    }
    const orders = await Order.aggregate([
      { $match: match },
      { $lookup: { from: 'orderitems', localField: '_id', foreignField: 'order', as: 'items' } },
      { $addFields: { items_count: { $size: '$items' } } },
      { $project: { items: 0 } },
      { $sort: { created_at: -1 } }
    ]);
    res.json(orders);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch orders' });
  }
}

export async function updateOrderStatus(req, res) {
  try {
    const { status, internal_notes } = req.body;
    const validStatuses = ['pending', 'confirmed', 'printing', 'shipped', 'delivered', 'cancelled'];
    if (!validStatuses.includes(status)) return res.status(400).json({ error: 'Invalid status' });
    const order = await Order.findById(req.params.id);
    if (!order) return res.status(404).json({ error: 'Order not found' });
    await Order.findByIdAndUpdate(req.params.id, { status, updated_at: new Date() });
    if (internal_notes !== undefined) {
      await Order.findByIdAndUpdate(req.params.id, { internal_notes });
    }
    await OrderStatusHistory.create({
      order: req.params.id,
      status,
      changed_by: req.user.role === 'admin' ? 'admin' : 'customer',
      notes: `Status changed to ${status}`,
    });
    res.json({ message: 'Order status updated', status });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update order status' });
  }
}

export async function cancelOrder(req, res) {
  try {
    const order = await Order.findOne({ _id: req.params.id, user: req.user.id });
    if (!order) return res.status(404).json({ error: 'Order not found' });
    if (order.status !== 'pending') return res.status(400).json({ error: 'Only pending orders can be cancelled' });
    await Order.findByIdAndUpdate(req.params.id, { status: 'cancelled', updated_at: new Date() });
    await OrderStatusHistory.create({
      order: req.params.id,
      status: 'cancelled',
      changed_by: 'customer',
      notes: 'Cancelled by customer',
    });
    res.json({ message: 'Order cancelled' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to cancel order' });
  }
}

export async function generateInvoice(req, res) {
  try {
    const order = await Order.findById(req.params.id);
    if (!order) return res.status(404).json({ error: 'Order not found' });
    if (req.user.role !== 'admin' && order.user?.toString() !== req.user.id) {
      return res.status(403).json({ error: 'Access denied' });
    }
    const items = await OrderItem.find({ order: order._id });
    const doc = new jsPDF();
    doc.setFontSize(20);
    doc.text('PrintShop', 14, 22);
    doc.setFontSize(10);
    doc.text('Invoice', 14, 30);
    doc.setFontSize(9);
    doc.text(`Order #: ${order.order_number}`, 14, 38);
    doc.text(`Date: ${new Date(order.created_at).toLocaleDateString()}`, 14, 44);
    doc.text(`Customer: ${order.customer_name}`, 14, 50);
    doc.text(`Email: ${order.customer_email}`, 14, 56);
    doc.text(`Phone: ${order.customer_phone}`, 14, 62);
    let y = 72;
    doc.setFontSize(10);
    doc.text('Items', 14, y);
    y += 6;
    doc.setFontSize(8);
    doc.setFillColor(240, 240, 240);
    doc.rect(14, y, 182, 5, 'F');
  }
}

```

```

doc.text('Product', 16, y + 4);
doc.text('Qty', 120, y + 4);
doc.text('Unit Price', 140, y + 4);
doc.text('Total', 175, y + 4);
y += 8;
for (const item of items) {
  doc.text(item.product_name.substring(0, 30), 16, y);
  doc.text(String(item.quantity), 120, y);
  doc.text(`¹ $ { item.unit_price.toFixed(2) }`, 140, y);
  doc.text(`¹ $ { item.line_total.toFixed(2) }`, 175, y);
  y += 5;
  if (y > 270) { doc.addPage(); y = 20; }
}
y += 4;
doc.line(14, y, 196, y);
y += 6;
doc.text(`Subtotal: ¹ $ { order.subtotal.toFixed(2) }`, 150, y);
y += 5;
if (order.coupon_discount > 0) {
  doc.text(`Discount ($ { order.coupon_code }): -¹ $ { order.coupon_discount.toFixed(2) }`, 150, y);
  y += 5;
  doc.text(`GST (18%): ¹ $ { order.gst_amount.toFixed(2) }`, 150, y);
  y += 5;
  doc.setFontSize(10);
  doc.text(`Total: ¹ $ { order.grand_total.toFixed(2) }`, 150, y);
  const pdfBuffer = Buffer.from(doc.output('arraybuffer'));
  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', 'attachment; filename=invoice-${order.order_number}.pdf');
  res.send(pdfBuffer);
} catch (err) {
  console.error('Invoice error:', err);
  res.status(500).json({ error: 'Failed to generate invoice' });
}
}

// File: server/controllers/pricingController.js (// Lines: 101)
// =====
import Product from '../models/Product.js';
import PricingRule from '../models/PricingRule.js';
import FinishUpcharge from '../models/FinishUpcharge.js';

export async function computePrice(req, res) {
  try {
    const productId = req.params.productId;
    const { size, finish, quantity: qtyParam, waterproof, sides, corners, weight } = req.query;
    const quantity = parseInt(qtyParam) || 1;
    const product = await Product.findById(productId);
    if (!product) return res.status(404).json({ error: 'Product not found' });
    const category = product.category;
    let basePrice = product.base_price;
    if (category === 'posters') {
      const sizeMultipliers = { 'A4': 1, 'A3': 1.4, 'A2': 2.2, 'A1': 3.4 };
      basePrice = product.base_price * (sizeMultipliers[size] || 1);
    }
    let upcharges = [];
    if (finish) {
      const upchargeRow = await FinishUpcharge.findOne({ product: productId, finish_name: finish });
      if (upchargeRow && upchargeRow.upcharge_amount > 0) {
        upcharges.push({ name: finish, amount: upchargeRow.upcharge_amount });
      }
    }
    const totalUpcharge = upcharges.reduce((sum, u) => sum + u.amount, 0);
    let effectiveBase = basePrice + totalUpcharge;
    if (category === 'stickers' && waterproof === 'true') effectiveBase = effectiveBase * 1.1;
    let extraNotes = [];
    if (category === 'visiting-cards') {
      if (sides === 'Double') { const d = 200 / quantity; effectiveBase += d; extraNotes.push({ name: 'Double-sided', amount: d }); }
      if (corners === 'Rounded') { const r = 150 / quantity; effectiveBase += r; extraNotes.push({ name: 'Rounded corners', amount: r }); }
    }
    const rule = await PricingRule.findOne({
      product: productId,
      min_qty: { $lte: quantity },
      max_qty: { $gte: quantity }
    });
    const discountPercent = rule ? rule.discount_percent : 0;
    const unitPrice = Math.round(effectiveBase * (1 - discountPercent / 100) * 100) / 100;
    const subtotal = Math.round(unitPrice * quantity * 100) / 100;
    const discountAmount = Math.round((effectiveBase * quantity - subtotal) * 100) / 100;
    res.json({
      product_id: parseInt(productId), product_name: product.name, category,
      base_price: basePrice,
      upcharges: [...upcharges, ...extraNotes],
      total_upcharge: Math.round(totalUpcharge * 100) / 100,
      effective_base: Math.round(effectiveBase * 100) / 100,
      discount_percent: discountPercent,
      discount_amount: discountAmount,
      unit_price: unitPrice,
      quantity,
      subtotal,
      turnaround_time: product.turnaround_time
    });
  } catch (err) {
    console.error('Pricing error:', err);
    res.status(500).json({ error: 'Failed to compute price' });
  }
}

```

```

export async function getPricingRules(req, res) {
  try {
    const [rules, upcharges] = await Promise.all([
      PricingRule.aggregate([
        { $lookup: { from: 'products', localField: 'product', foreignField: '_id', as: 'product' } },
        { $unwind: '$product' },
        { $sort: { 'product.name': 1, min_qty: 1 } },
        { $project: { id: '$_id', product_id: '$product._id', product_name: '$product.name', category: '$product.category', min_qty: 1, max_qty: 1, discount_percent: 1 } }
      ]),
      FinishUpcharge.aggregate([
        { $lookup: { from: 'products', localField: 'product', foreignField: '_id', as: 'product' } },
        { $unwind: '$product' },
        { $sort: { 'product.name': 1, finish_name: 1 } },
        { $project: { id: '$_id', product_id: '$product._id', product_name: '$product.name', category: '$product.category', finish_name: 1, upcharge_amount: 1 } }
      ]),
    ]);
    res.json({ rules, upcharges });
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch pricing rules' });
  }
}

export async function updatePricingRules(req, res) {
  try {
    const { rules, upcharges } = req.body;
    if (rules) {
      await Promise.all(rules.map(rule =>
        PricingRule.findByIdAndUpdate(rule.id, { discount_percent: rule.discount_percent })
      ));
    }
    if (upcharges) {
      await Promise.all(upcharges.map(u =>
        FinishUpcharge.findByIdAndUpdate(u.id, { upcharge_amount: u.upcharge_amount })
      ));
    }
    res.json({ message: 'Pricing rules updated' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update pricing rules' });
  }
}

// File: server/controllers/productController.js (// Lines: 106)
// =====
import Product from '../models/Product.js';
import PricingRule from '../models/PricingRule.js';
import Review from '../models/Review.js';

export async function listProducts(req, res) {
  try {
    const { category, status, sort, minPrice, maxPrice, finish } = req.query;
    const filter = {};
    if (category) filter.category = category;
    if (status) filter.status = status;
    else filter.status = 'active';
    if (minPrice || maxPrice) {
      filter.base_price = {};
      if (minPrice) filter.base_price.$gte = Number(minPrice);
      if (maxPrice) filter.base_price.$lte = Number(maxPrice);
    }
    if (finish) filter.available_finishes = { $regex: finish, $options: 'i' };
    const allowedSort = {
      price_asc: { base_price: 1 },
      price_desc: { base_price: -1 },
      popular: { order_count: -1 },
      newest: { created_at: -1 },
      rating: { avg_rating: -1 }
    };
    const sortOption = allowedSort[sort] || { order_count: -1 };
    const products = await Product.find(filter).sort(sortOption);
    res.json(products);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch products' });
  }
}

export async function getProduct(req, res) {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ error: 'Product not found' });
    res.json(product.toObject());
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch product' });
  }
}

export async function createProduct(req, res) {
  try {
    const { name, category, description, base_price, turnaround_time, available_sizes, available_finishes, status } = req.body;
    const product = await Product.create({
      name,
      category,
      description: description || '',
      base_price: Number(base_price),
      turnaround_time: turnaround_time || '3-5 business days',
      available_sizes: available_sizes || [],
      available_finishes: available_finishes || [],
    });
  }
}

```



```

    status: status || 'active',
  });
  const pid = product._id;
  const defaultRules = [
    { product: pid, min_qty: 1, max_qty: 4, discount_percent: 0 },
    { product: pid, min_qty: 5, max_qty: 9, discount_percent: 10 },
    { product: pid, min_qty: 10, max_qty: 24, discount_percent: 18 },
    { product: pid, min_qty: 25, max_qty: 999999, discount_percent: 25 },
  ];
  await PricingRule.insertMany(defaultRules);
  res.status(201).json({ id: pid.toString(), message: 'Product created' });
} catch (err) {
  res.status(500).json({ error: 'Failed to create product' });
}
}

export async function updateProduct(req, res) {
  try {
    const { name, category, description, base_price, turnaround_time, available_sizes, available_finishes } = req.body;
    await Product.findByIdAndUpdate(req.params.id, {
      name, category, description, base_price: Number(base_price), turnaround_time,
      available_sizes: available_sizes || [],
      available_finishes: available_finishes || [],
    });
    res.json({ message: 'Product updated' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update product' });
  }
}

export async function toggleProductStatus(req, res) {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ error: 'Product not found' });
    const newStatus = product.status === 'active' ? 'inactive' : 'active';
    await Product.findByIdAndUpdate(req.params.id, { status: newStatus });
    res.json({ status: newStatus });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update status' });
  }
}

export async function getProductReviews(req, res) {
  try {
    const reviews = await Review.find({ product: req.params.id })
      .populate('user', 'name')
      .sort({ created_at: -1 });
    res.json(reviews);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch reviews' });
  }
}

// File: server/controllers/reviewController.js (// Lines: 99)
// =====
import Review from '../models/Review.js';
import OrderItem from '../models/OrderItem.js';
import Order from '../models/Order.js';
import Product from '../models/Product.js';

export async function createReview(req, res) {
  try {
    const { product_id, order_item_id, rating, comment } = req.body;
    const orderItem = await OrderItem.findById(order_item_id).populate('order');
    if (!orderItem) return res.status(404).json({ error: 'Order item not found' });
    if (orderItem.order.user?.toString() !== req.user.id) return res.status(403).json({ error: 'Not your order' });
    if (orderItem.order.status !== 'delivered') return res.status(400).json({ error: 'Can only review delivered items' });
    const existing = await Review.findOne({ order_item: order_item_id, user: req.user.id });
    if (existing) return res.status(400).json({ error: 'Already reviewed this item' });
    await Review.create({
      product: product_id,
      user: req.user.id,
      order_item: order_item_id,
      rating,
      comment: comment || '',
    });
    const stats = await Review.aggregate([
      { $match: { product: product_id } },
      { $group: { _id: null, avg: { $avg: '$rating' }, count: { $sum: 1 } } }
    ]);
    const newAvg = stats[0]?.avg || rating;
    const newTotal = stats[0]?.count || 1;
    await Product.findByIdAndUpdate(product_id, {
      avg_rating: Math.round(newAvg * 10) / 10,
      total_reviews: newTotal,
    });
    res.status(201).json({ message: 'Review submitted' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to submit review' });
  }
}

export async function updateReview(req, res) {
  try {
    const review = await Review.findOne({ _id: req.params.id, user: req.user.id });
    if (!review) return res.status(404).json({ error: 'Review not found' });
    const { rating, comment } = req.body;
    await Review.findByIdAndUpdate(req.params.id, { rating, comment });
  }
}

```

```

const stats = await Review.aggregate([
  { $match: { product: review.product } },
  { $group: { _id: null, avg: { $avg: '$rating' }, count: { $sum: 1 } } }
]);
await Product.findByIdAndUpdate(review.product, {
  avg_rating: Math.round((stats[0]?.avg || rating) * 10) / 10,
  total_reviews: stats[0]?.count || 1,
});
res.json({ message: 'Review updated' });
} catch (err) {
  res.status(500).json({ error: 'Failed to update review' });
}
}

export async function getDeliveredItemsWithoutReview(req, res) {
  try {
    const items = await OrderItem.aggregate([
      { $lookup: { from: 'orders', localField: 'order', foreignField: '_id', as: 'order' } },
      { $unwind: '$order' },
      { $match: { 'order.user': req.user.id, 'order.status': 'delivered' } },
      { $lookup: { from: 'reviews', localField: '_id', foreignField: 'order_item', as: 'reviews' } },
      { $match: { reviews: { $size: 0 } } },
      { $project: {
        id: '$_id',
        product_id: '$product',
        product_name: 1,
        category: 1,
        configuration: 1,
        quantity: 1,
        order_number: '$order.order_number',
        created_at: '$order.created_at'
      } },
      { $sort: { created_at: -1 } }
    ]);
    for (const item of items) {
      if (typeof item.configuration === 'string') {
        try { item.configuration = JSON.parse(item.configuration); } catch (e) { }
      }
    }
    res.json(items);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch items' });
  }
}

export async function getUserReviews(req, res) {
  try {
    const reviews = await Review.find({ user: req.user.id })
      .populate('product', 'name category')
      .sort({ created_at: -1 });
    res.json(reviews);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch reviews' });
  }
}

// File: server/routes/addresses.js (// Lines: 13)
// =====
import { Router } from 'express';
import { listAddresses, createAddress, updateAddress, deleteAddress } from '../controllers/addressController.js';
import { authenticate } from '../middleware/auth.js';

const router = Router();

router.get('/', authenticate, listAddresses);
router.post('/', authenticate, createAddress);
router.put('/:id', authenticate, updateAddress);
router.delete('/:id', authenticate, deleteAddress);

export default router;

// File: server/routes/auth.js (// Lines: 12)
// =====
import { Router } from 'express';
import { register, login, getMe } from '../controllers/authController.js';
import { authenticate } from '../middleware/auth.js';

const router = Router();

router.post('/register', register);
router.post('/login', login);
router.get('/me', authenticate, getMe);

export default router;

// File: server/routes/coupons.js (// Lines: 14)
// =====
import { Router } from 'express';
import { validateCoupon, listCoupons, createCoupon, updateCoupon, deleteCoupon } from '../controllers/couponController.js';
import { authenticate, requireAdmin } from '../middleware/auth.js';

const router = Router();

router.post('/validate', validateCoupon);
router.get('/', authenticate, requireAdmin, listCoupons);
router.post('/', authenticate, requireAdmin, createCoupon);
router.put('/:id', authenticate, requireAdmin, updateCoupon);

```

```

router.delete('/:id', authenticate, requireAdmin, deleteCoupon);

export default router;

// File: server/routes/dashboard.js (// Lines: 10)
// =====
import { Router } from 'express';
import { getAdminDashboard } from '../controllers/dashboardController.js';
import { authenticate, requireAdmin } from '../middleware/auth.js';

const router = Router();

router.get('/admin', authenticate, requireAdmin, getAdminDashboard);

export default router;

// File: server/routes/orders.js (// Lines: 16)
// =====
import { Router } from 'express';
import { createOrder, getMyOrders, getOrder, listAllOrders, updateOrderStatus, cancelOrder, generateInvoice } from '../controllers/orderController.js';
import { authenticate, requireAdmin } from '../middleware/auth.js';

const router = Router();

router.post('/', authenticate, createOrder);
router.get('/my', authenticate, getMyOrders);
router.get('/', authenticate, requireAdmin, listAllOrders);
router.get('/:id', authenticate, getOrder);
router.patch('/:id/status', authenticate, requireAdmin, updateOrderStatus);
router.patch('/:id/cancel', authenticate, cancelOrder);
router.get('/:id/invoice', authenticate, generateInvoice);

export default router;

// File: server/routes/pricing.js (// Lines: 12)
// =====
import { Router } from 'express';
import { computePrice, getPricingRules, updatePricingRules } from '../controllers/pricingController.js';
import { authenticate, requireAdmin } from '../middleware/auth.js';

const router = Router();

router.get('/:productId', computePrice);
router.get('/', authenticate, requireAdmin, getPricingRules);
router.put('/', authenticate, requireAdmin, updatePricingRules);

export default router;

// File: server/routes/products.js (// Lines: 15)
// =====
import { Router } from 'express';
import { listProducts, getProduct, createProduct, updateProduct, toggleProductStatus, getProductReviews } from '../controllers/productController.js';
import { authenticate, requireAdmin } from '../middleware/auth.js';

const router = Router();

router.get('/', listProducts);
router.get('/:id', getProduct);
router.get('/:id/reviews', getProductReviews);
router.post('/', authenticate, requireAdmin, createProduct);
router.put('/:id', authenticate, requireAdmin, updateProduct);
router.patch('/:id/status', authenticate, requireAdmin, toggleProductStatus);

export default router;

// File: server/routes/reviews.js (// Lines: 13)
// =====
import { Router } from 'express';
import { createReview, updateReview, getDeliveredItemsWithoutReview, getUserReviews } from '../controllers/reviewController.js';
import { authenticate } from '../middleware/auth.js';

const router = Router();

router.post('/', authenticate, createReview);
router.put('/:id', authenticate, updateReview);
router.get('/pending', authenticate, getDeliveredItemsWithoutReview);
router.get('/my', authenticate, getUserReviews);

export default router;

// File: server/routes/upload.js (// Lines: 25)
// =====
import { Router } from 'express';
import { upload } from '../middleware/upload.js';
import { authenticate } from '../middleware/auth.js';

const router = Router();

router.post('/design', authenticate, (req, res) => {
  upload.single('design')(req, res, (err) => {
    if (err) {
      return res.status(400).json({ error: err.message });
    }
  })
});

```

```
if (!req.file) {
  return res.status(400).json({ error: 'No file uploaded' });
}
res.json({
  file_path: req.file.filename,
  original_name: req.file.originalname,
  size: req.file.size,
  url: `/uploads/${req.file.filename}`
});
});
});

export default router;
```